

# Hundred Day's of Coding C programming

## Experiment-4

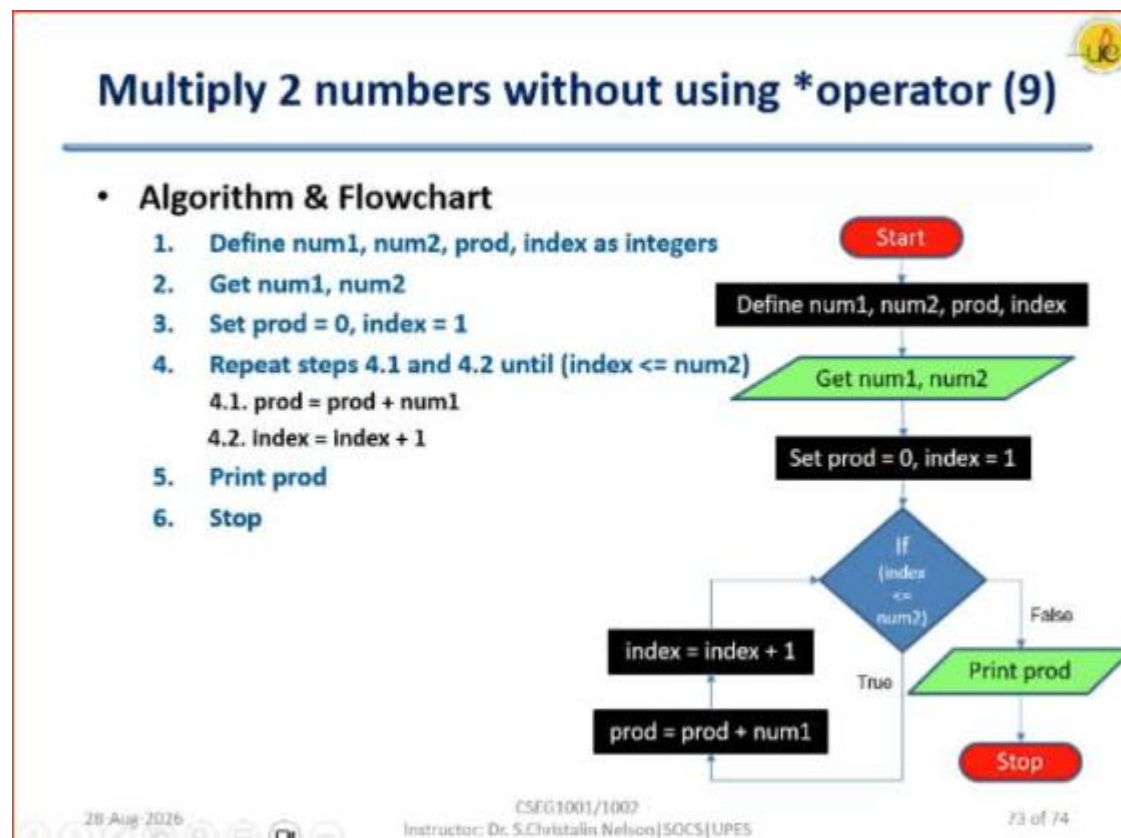
**Name:** Arayana Khare

**Batch:** 15

**Sap Id:** 590042432

### Question

Find the incomplete algorithm & flowchart attaches below for today's HDOC Question.



### Question 1

Identify the mistakes in the given algorithm.

#### **Mistakes in the given algorithm**

I found a few mistakes in the given algorithm:

**1. Mistake in Step 4:**

It says “**Repeat until (index <= num2)**”. This is not correct because the loop should continue as long as index is less than or equal to num2.

So, it should be “**Repeat while (index <= num2)**”.

**2. The flowchart is slightly confusing:**

In the flowchart, after checking  $\text{index} \leq \text{num2}$ , the steps are shown as  $\text{index} = \text{index} + 1$  followed by  $\text{prod} = \text{prod} + \text{num1}$ . It would be clearer to first add num1 to prod and then increase the index.

**3. Negative numbers are not considered:**

The algorithm works properly for positive numbers, but it will not give the correct answer if num2 is negative.

**Question 2**

**4. Corrected Step 4**

**Repeat steps 4.1 and 4.2 while (index <= num2)**

**4.1**  $\text{prod} = \text{prod} + \text{num1}$

**4.2**  $\text{index} = \text{index} + 1$

**Question 3**

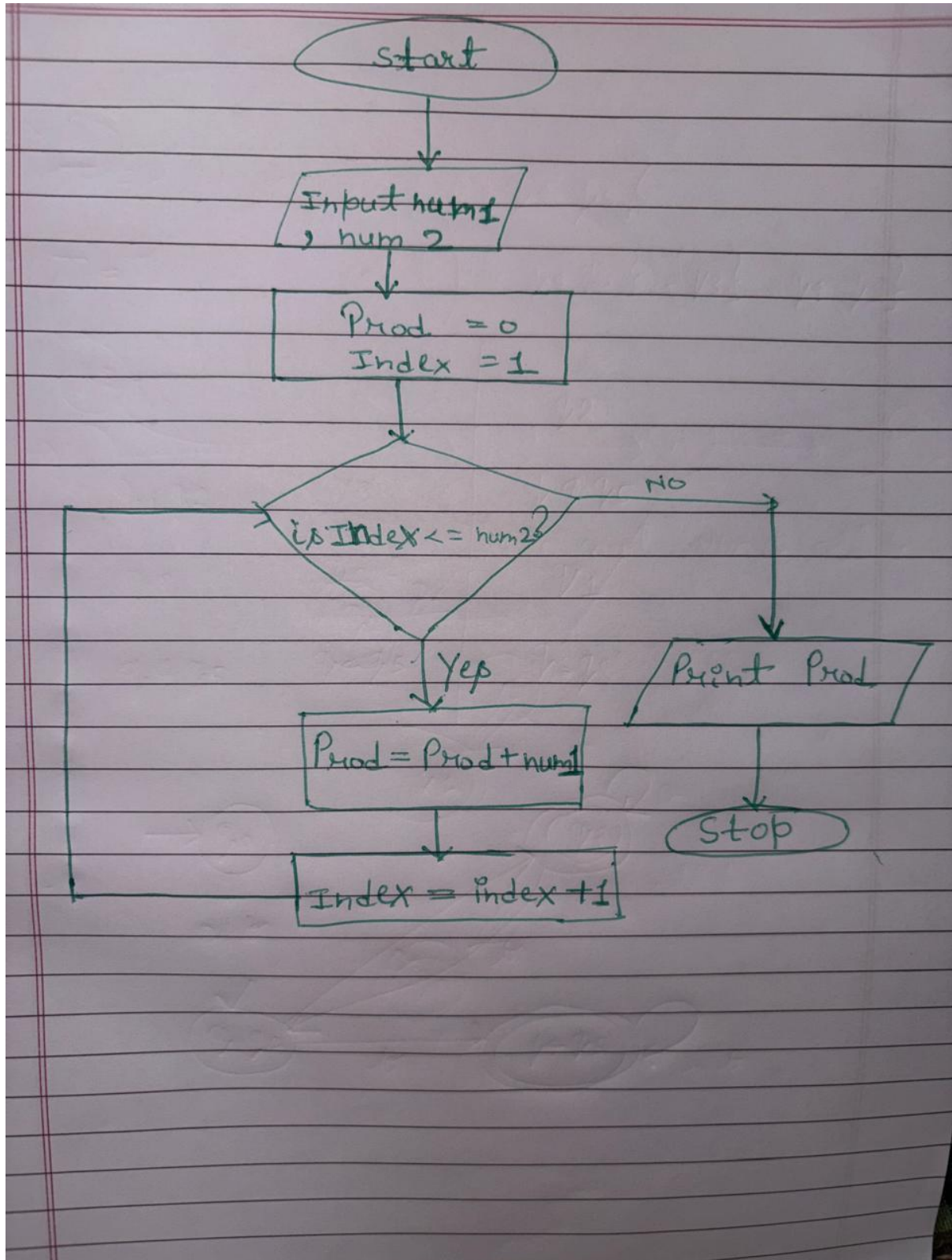
**Find the mistakes in the flow chart.**

1. The flowchart does not handle negative numbers properly.
2. It does not handle negative \* negative correctly.
3. The condition  $\text{index} \leq \text{num2}$  does not work when num2 is negative.
4. There is no proper check for zero.
5. It does not always use the minimum number of additions.
6. It always uses num2 for repetition, instead of choosing the smaller number for fewer additions.

#### Question 4

#### Correct Flow Chart

Corrected flow chart.



## **Question**

The following problem is to find the product of two integers without using the multiplication operator  $*$ .

Your teacher has given an incorrect algorithm and an incorrect flowchart for this problem (Refer Slideshare).

Study both carefully and complete the tasks given below. [Hint: Use repeated addition]

Requirements:

The logic must use the minimum possible number of additions and should correctly handle:

- positive  $\times$  positive
- positive  $\times$  negative
- negative  $\times$  positive
- negative  $\times$  negative
- multiplication by zero

## **Tasks**

- Identify the mistakes in the given algorithm.
- Rewrite the corrected algorithm.
- Identify the mistakes in the given flowchart.
- Redraw the corrected flowchart.
- Verify your corrected algorithm and flow chart using the given test cases.

| Test Case | Input | Expected Output | Minimum Additions |
|-----------|-------|-----------------|-------------------|
| 1         | 12 3  | Product: 36     | 3                 |
| 2         | 3 12  | Product: 36     | 3                 |
| 3         | -6 5  | Product: -30    | 5                 |
| 4         | 6 -5  | Product: -30    | 5                 |
| 5         | -9 -4 | Product: 36     | 4                 |
| 6         | -4 -9 | Product: 36     | 4                 |
| 7         | 0 8   | Product: 0      | 0                 |
| 8         | 8 0   | Product: 0      | 0                 |
| 9         | 0 -7  | Product: 0      | 0                 |
| 10        | -7 0  | Product: 0      | 0                 |
| 11        | 0 0   | Product: 0      | 0                 |
| 12        | 1 -7  | Product: -7     | 1                 |
| 13        | -1 -9 | Product: 9      | 1                 |

main.c

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int num1, num2;
6      int a, b;
7      int prod = 0;
8      int sign = 1;
9      int count;
10
11     printf("Enter two integers: ");
12     scanf("%d %d", &num1, &num2);
13
14     if (num1 == 0 || num2 == 0)
15     {
16         printf("Product: 0\n");
17         return 0;
18     }
19
20     a = num1;
21     b = num2;
22
23     if (a < 0)
24     {
25         sign = -sign;
26         a = -a;
27     }
28
29     if (b < 0)
30     {
31         sign = -sign;
32         b = -b;
33     }
34
35     if (a > b)
36     {
37         int temp = a;
38         a = b;
39         b = temp;
40     }
41
42     count = 0;
43
44     while (count < a)
45     {
```

```
46         prod = prod + b;
47         count = count + 1;
48     }
49
50     if (sign < 0)
51         prod = -prod;
52
53     printf("Product: %d\n", prod);
54
55     return 0;
56 }
```

### Test Case Explanation

#### **Case 1: 12 3**

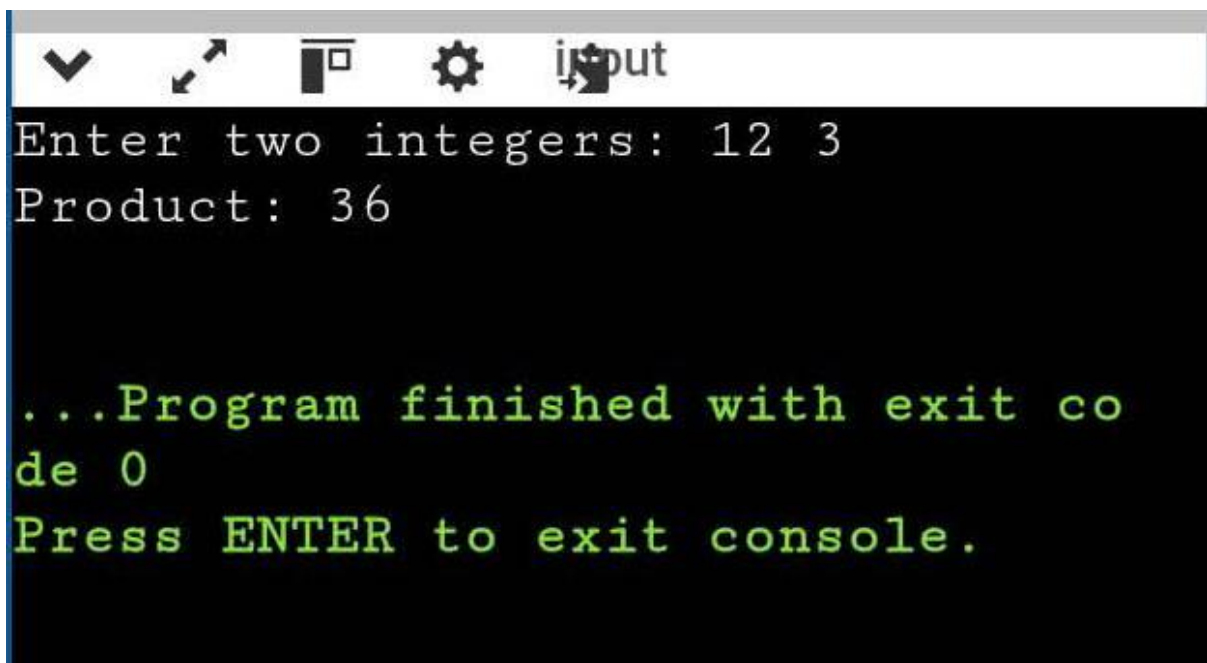
Both numbers are positive.

Smaller number is 3, so we add 12 three times.

$12 + 12 + 12 = 36$

Output: 36

Additions: 3

A screenshot of a console window with a dark background. The title bar is light gray and contains icons for a dropdown menu, a window, a settings gear, and a cursor icon. The text in the console is as follows:  
Enter two integers: 12 3  
Product: 36  
  
...Program finished with exit code 0  
Press ENTER to exit console.

### Case 2: 3 12

Both numbers are positive.

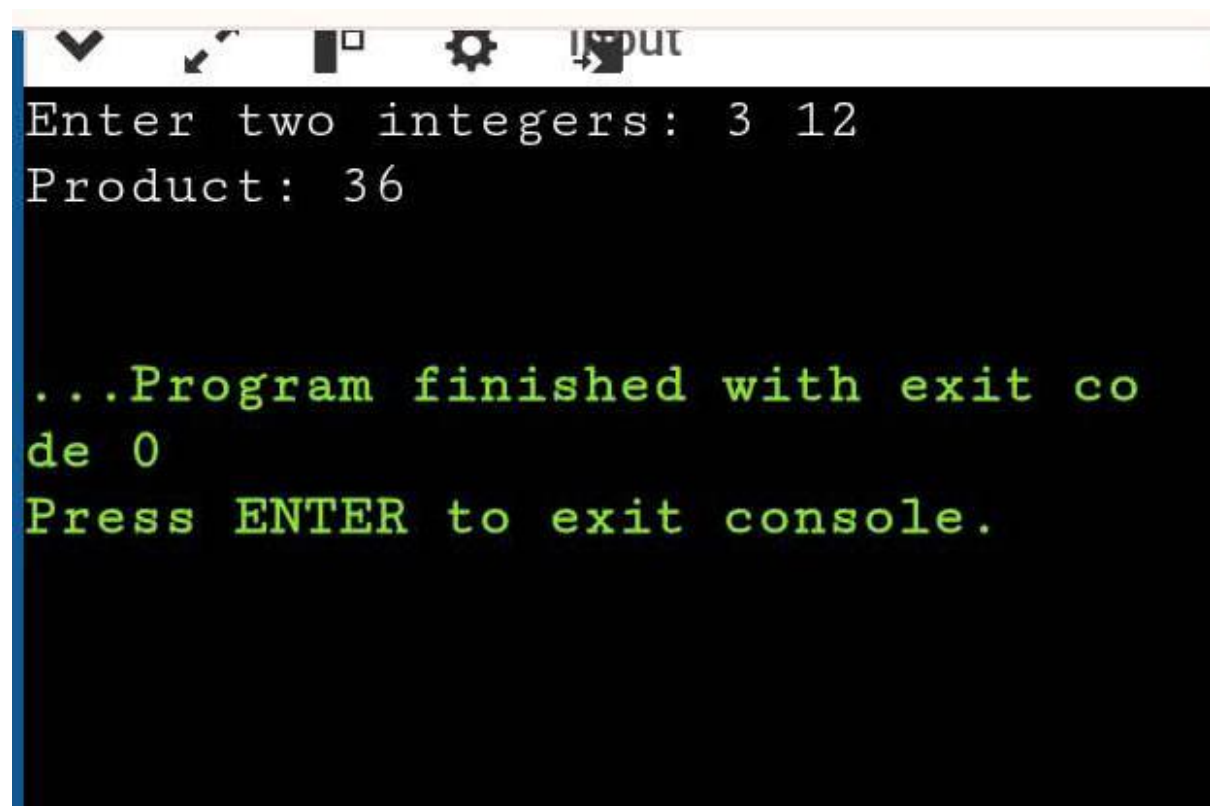
Smaller number is 3.

We add 12 three times.

$$12 + 12 + 12 = 36$$

Output: 36

Additions: 3



```
Enter two integers: 3 12
Product: 36

...Program finished with exit co
de 0
Press ENTER to exit console.
```

### Case 3: -6 5

First number is negative.

We take its positive value as 6 and remember that the answer should be negative.

We add 6 five times.

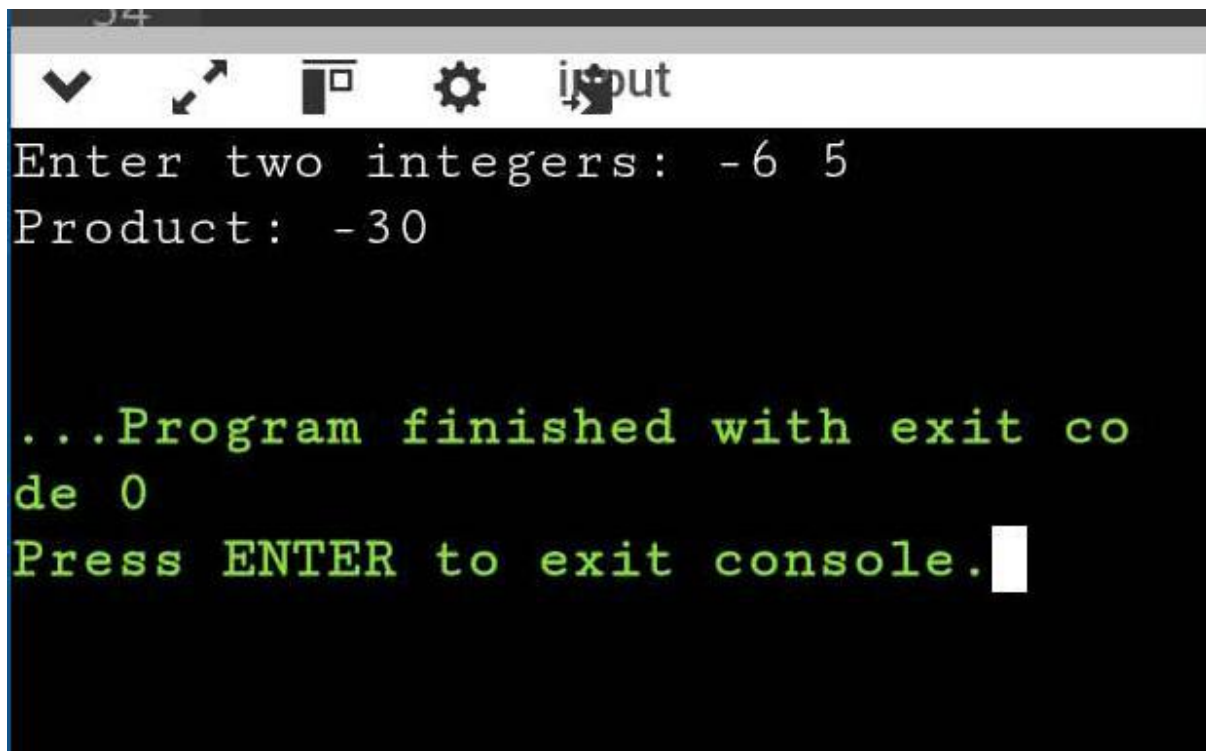
$$6 + 6 + 6 + 6 + 6 = 30$$

So the answer is -30.



Output: -30

Additions: 5

A screenshot of a console window with a dark background and a light gray title bar. The title bar contains several icons: a downward arrow, a diagonal arrow, a square icon, a gear icon, and the word 'input'. The console text is as follows:  
Enter two integers: -6 5  
Product: -30  
  
...Program finished with exit code 0  
Press ENTER to exit console.  
The text is in a monospaced font. The first two lines are white, and the last three lines are green. A white cursor is visible at the end of the last line.

#### Case 4: 6 -5

Second number is negative.

We take its positive value as 5 and keep the negative sign.

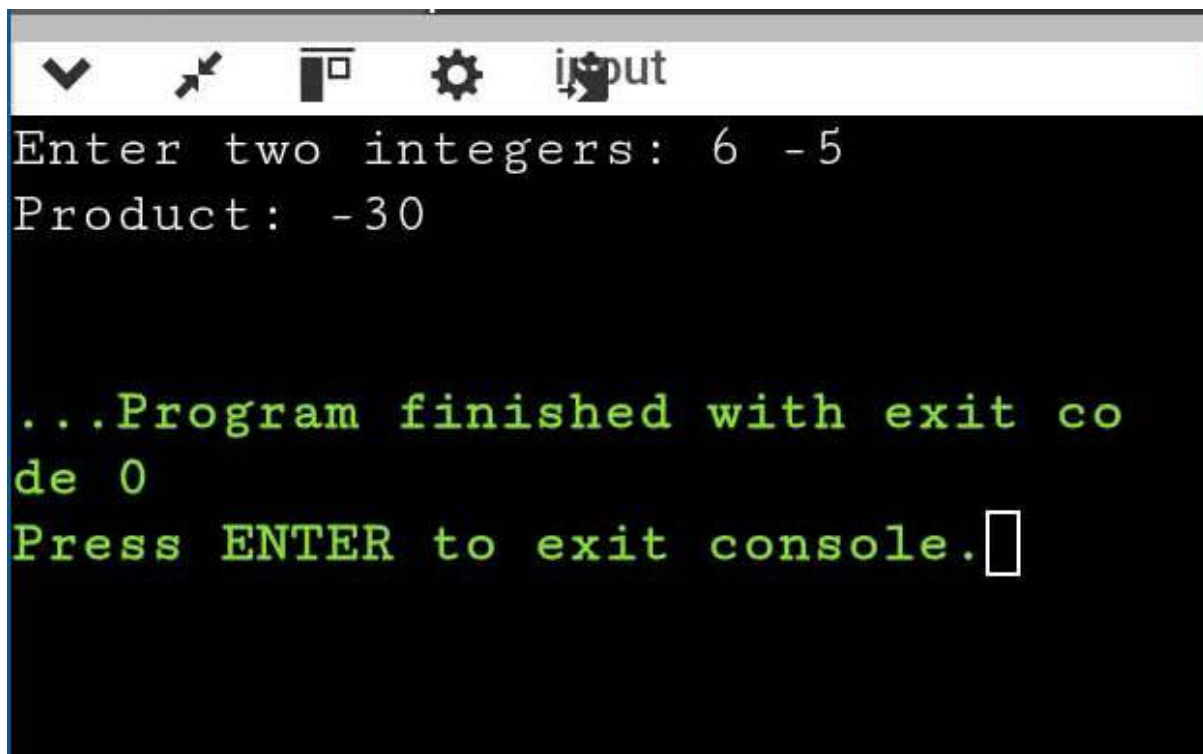
We add 6 five times.

$$6 + 6 + 6 + 6 + 6 = 30$$

Final answer is negative.

Output: -30

Additions: 5



```
Enter two integers: 6 -5
Product: -30

...Program finished with exit co
de 0
Press ENTER to exit console.
```

#### Case 5: -9 -4

Both numbers are negative.

Negative signs cancel each other, so the answer will be positive.

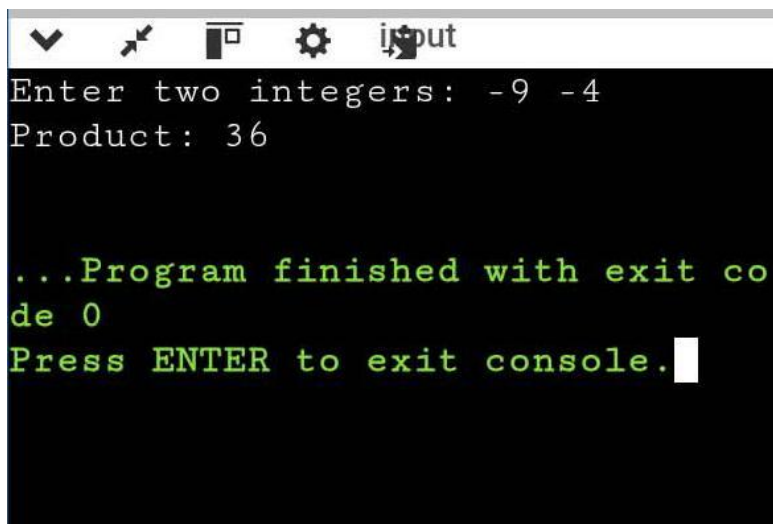
Smaller number is 4.

We add 9 four times.

$$9 + 9 + 9 + 9 = 36$$

Output: 36

Additions: 4



```
Enter two integers: -9 -4
Product: 36

...Program finished with exit co
de 0
Press ENTER to exit console.
```

### Case 6: -4 -9

Both numbers are negative, so the answer will be positive.

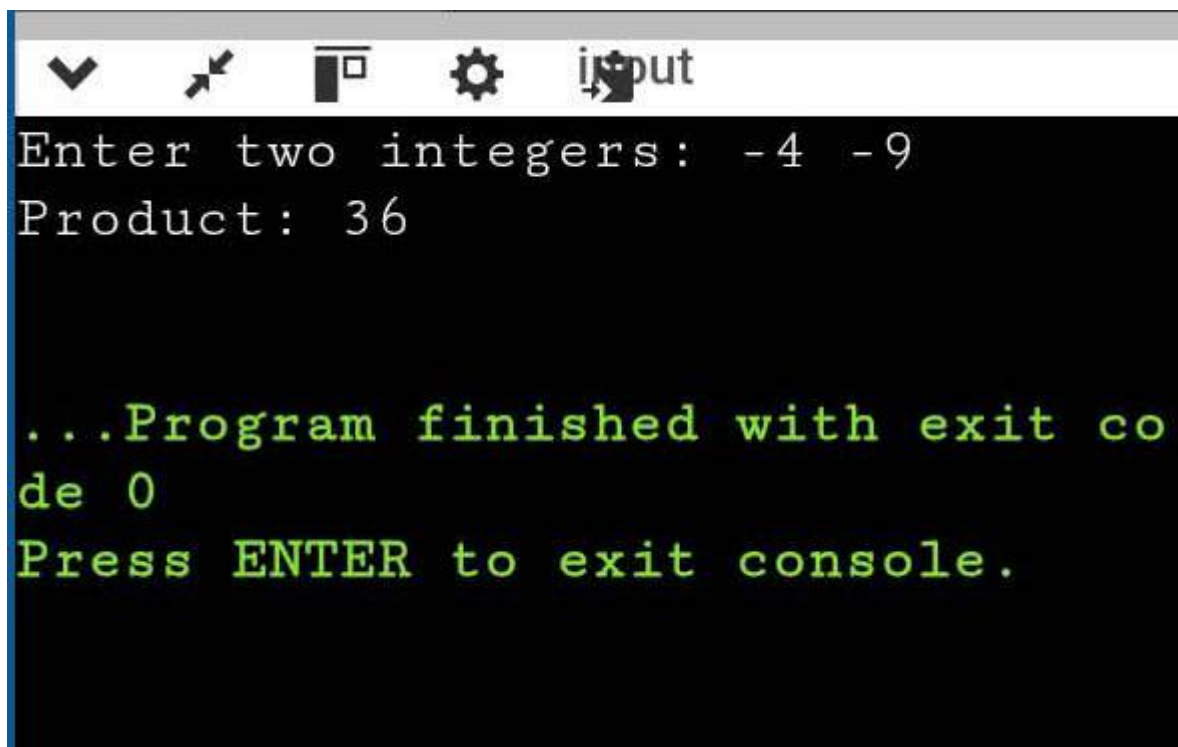
Smaller number is 4.

We add 9 four times.

$$9 + 9 + 9 + 9 = 36$$

Output: 36

Additions: 4

A screenshot of a console window with a dark background and light green text. The window has a title bar with standard icons (minimize, maximize, close) and the word 'input'. The text inside the console reads: 'Enter two integers: -4 -9', 'Product: 36', followed by a blank line, then '...Program finished with exit code 0', and finally 'Press ENTER to exit console.'.

```
Enter two integers: -4 -9
Product: 36

...Program finished with exit code 0
Press ENTER to exit console.
```

### Case 7: 0 8

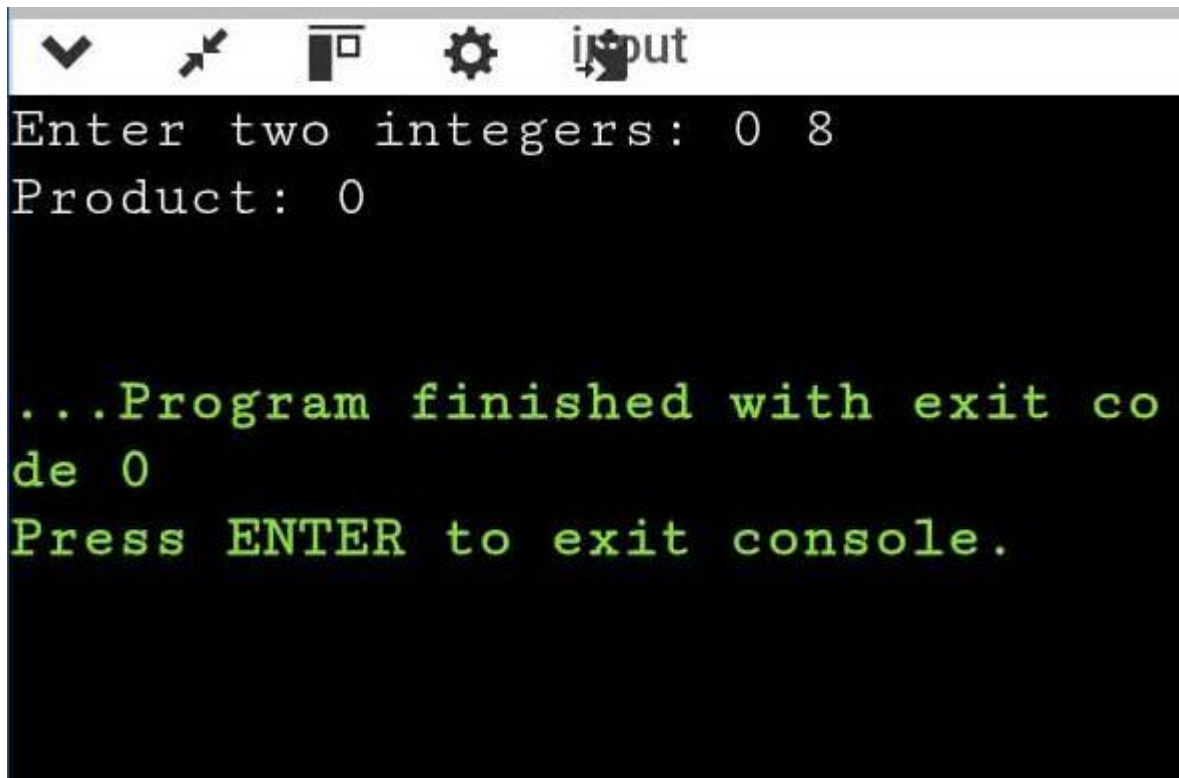
One number is zero.

Any number multiplied by zero gives zero.

So no addition is required.

Output: 0

Additions: 0



```
Enter two integers: 0 8
Product: 0

...Program finished with exit co
de 0
Press ENTER to exit console.
```

#### Case 8: 8 0

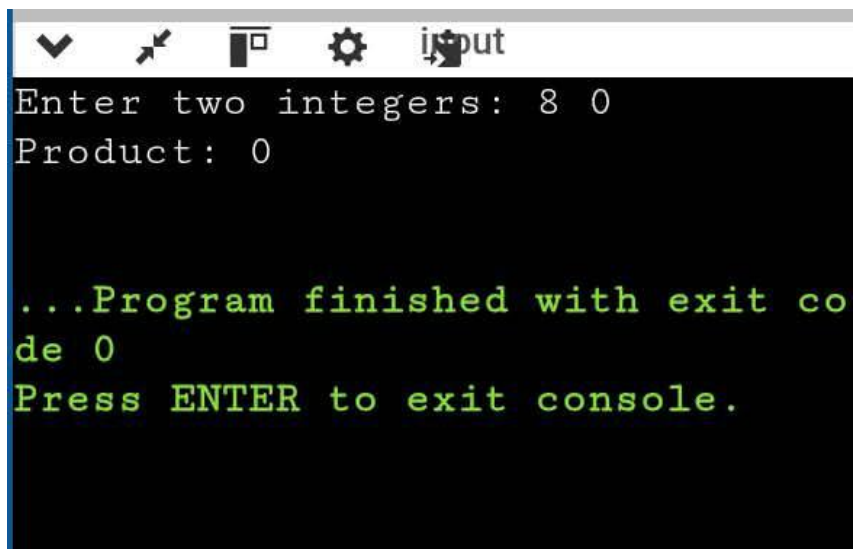
Second number is zero.

Therefore, the product is zero.

No loop or addition is required.

Output: 0

Additions: 0



```
Enter two integers: 8 0
Product: 0

...Program finished with exit co
de 0
Press ENTER to exit console.
```

### Case 9: 0 -7

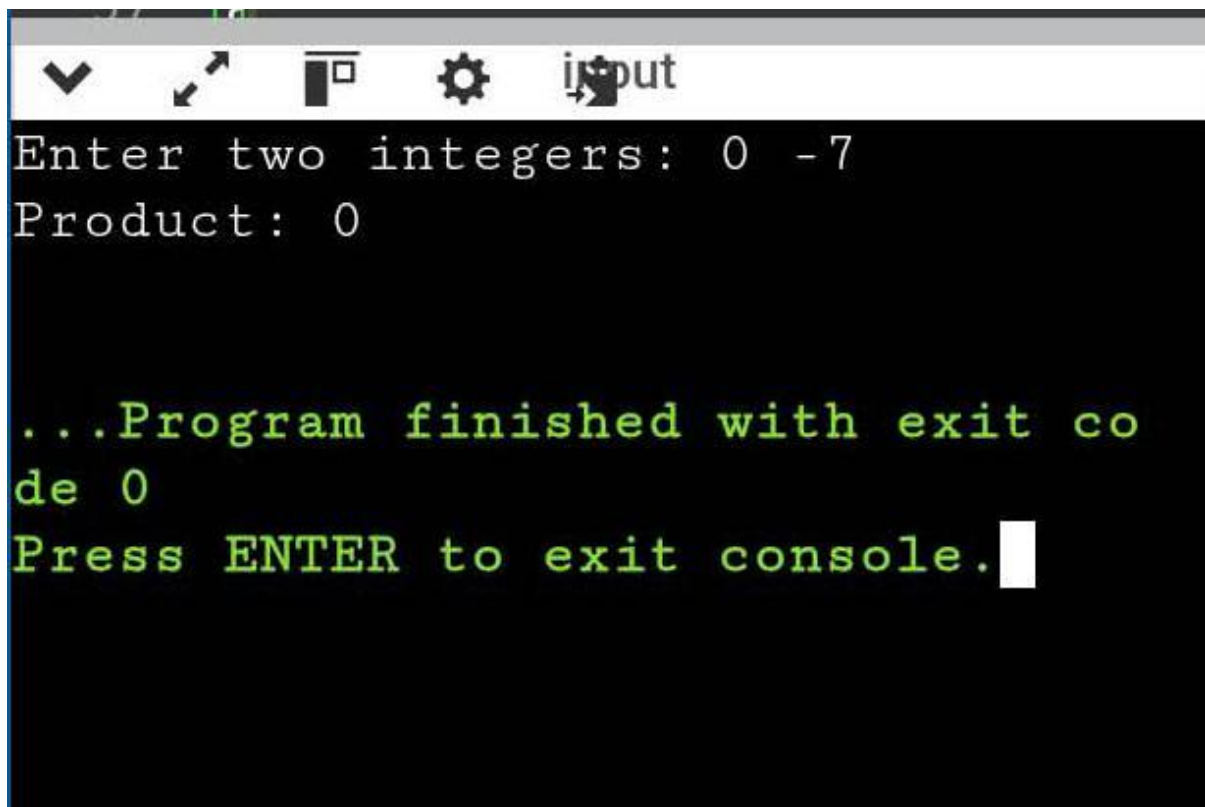
First number is zero.

The other number does not matter.

Product is zero.

Output: 0

Additions: 0

A screenshot of a console window with a dark background and light green text. The window has a title bar with standard icons (checkmark, cursor, window, gear, and a button labeled 'input'). The text in the console reads: 'Enter two integers: 0 -7', 'Product: 0', followed by a blank line, then '...Program finished with exit code 0', and finally 'Press ENTER to exit console.' with a white cursor block at the end of the last line.

```
Enter two integers: 0 -7
Product: 0

...Program finished with exit code 0
Press ENTER to exit console.
```

### Case 10: -7 0

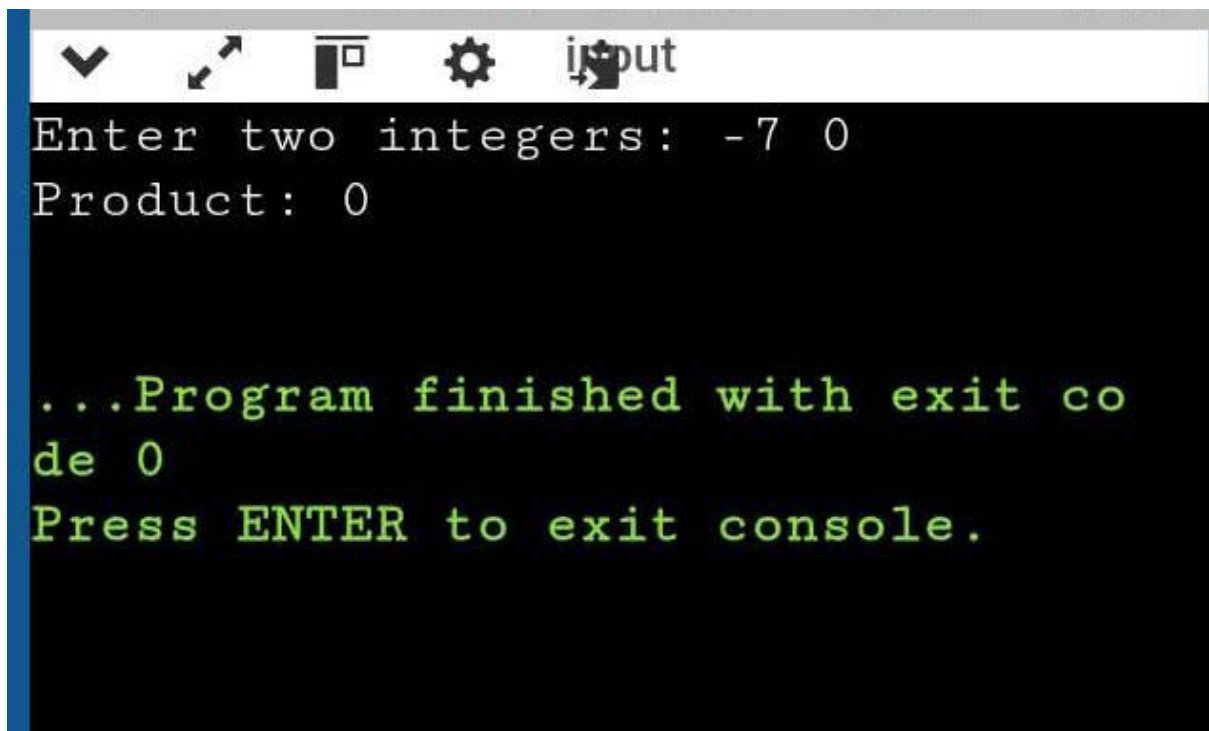
Second number is zero.

Therefore, the product is zero.

No addition is required.

Output: 0

Additions: 0

A screenshot of a console window with a dark background and a light blue title bar. The title bar contains several icons: a downward arrow, a cursor, a square, a gear, and the word 'input'. The console text is as follows:

```
Enter two integers: -7 0
Product: 0

...Program finished with exit co
de 0
Press ENTER to exit console.
```

#### Case 11: 0 0

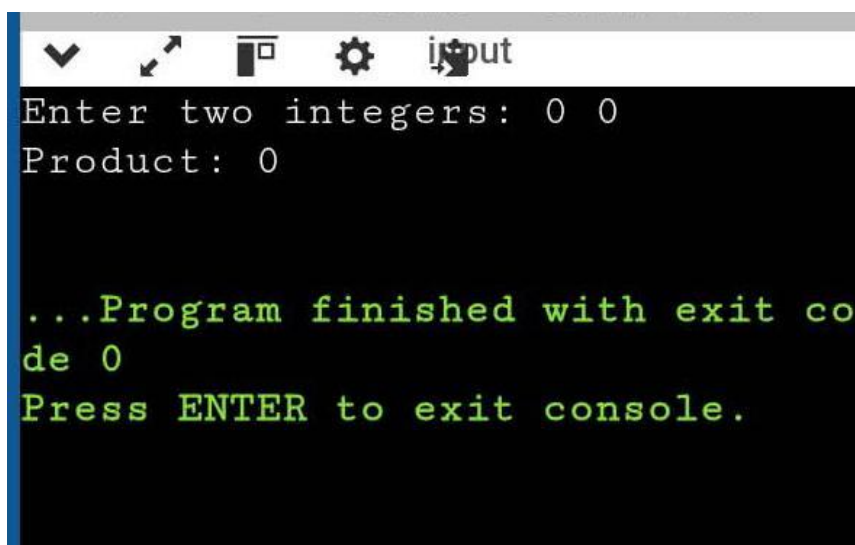
Both numbers are zero.

Product is zero.

No addition is required.

Output: 0

Additions: 0

A screenshot of a console window with a dark background and a light blue title bar. The title bar contains several icons: a downward arrow, a cursor, a square, a gear, and the word 'input'. The console text is as follows:

```
Enter two integers: 0 0
Product: 0

...Program finished with exit co
de 0
Press ENTER to exit console.
```

### Case 12: 1 -7

Second number is negative.

We use 7 as the positive value and remember the negative sign.

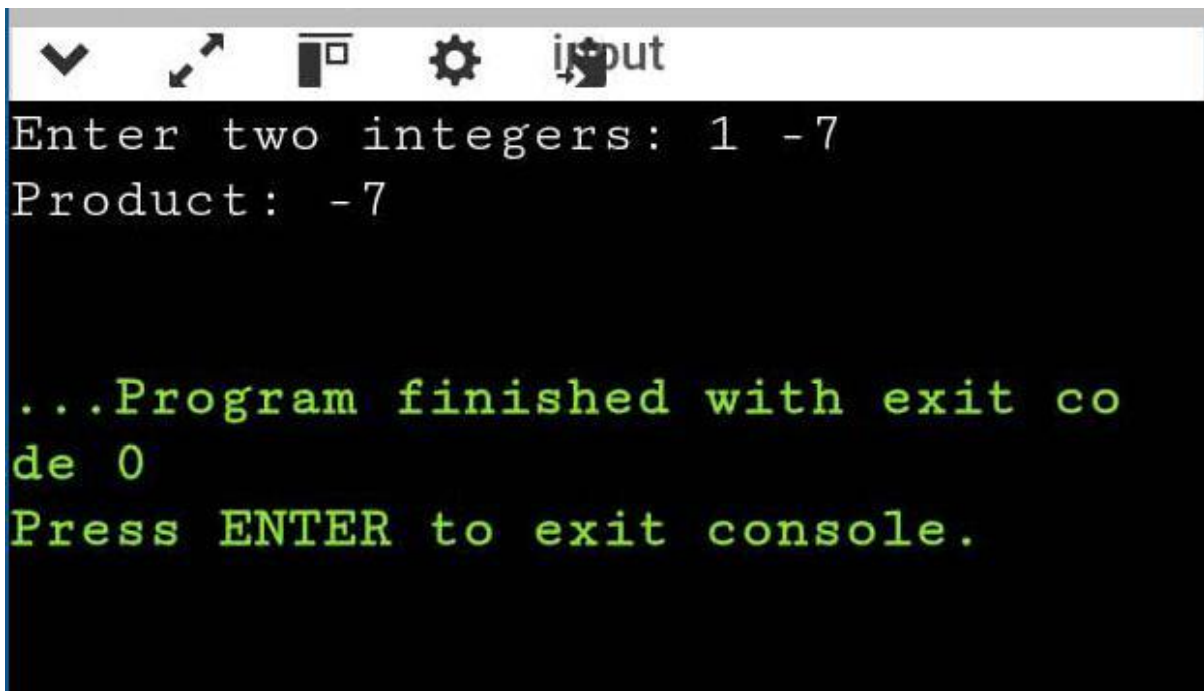
Smaller number is 1, so only one addition is needed.

$7 = 7$

Final answer is -7.

Output: -7

Additions: 1



```
Enter two integers: 1 -7
Product: -7

...Program finished with exit code 0
Press ENTER to exit console.
```

### Case 13: -1 -9

Both numbers are negative, so the answer is positive.

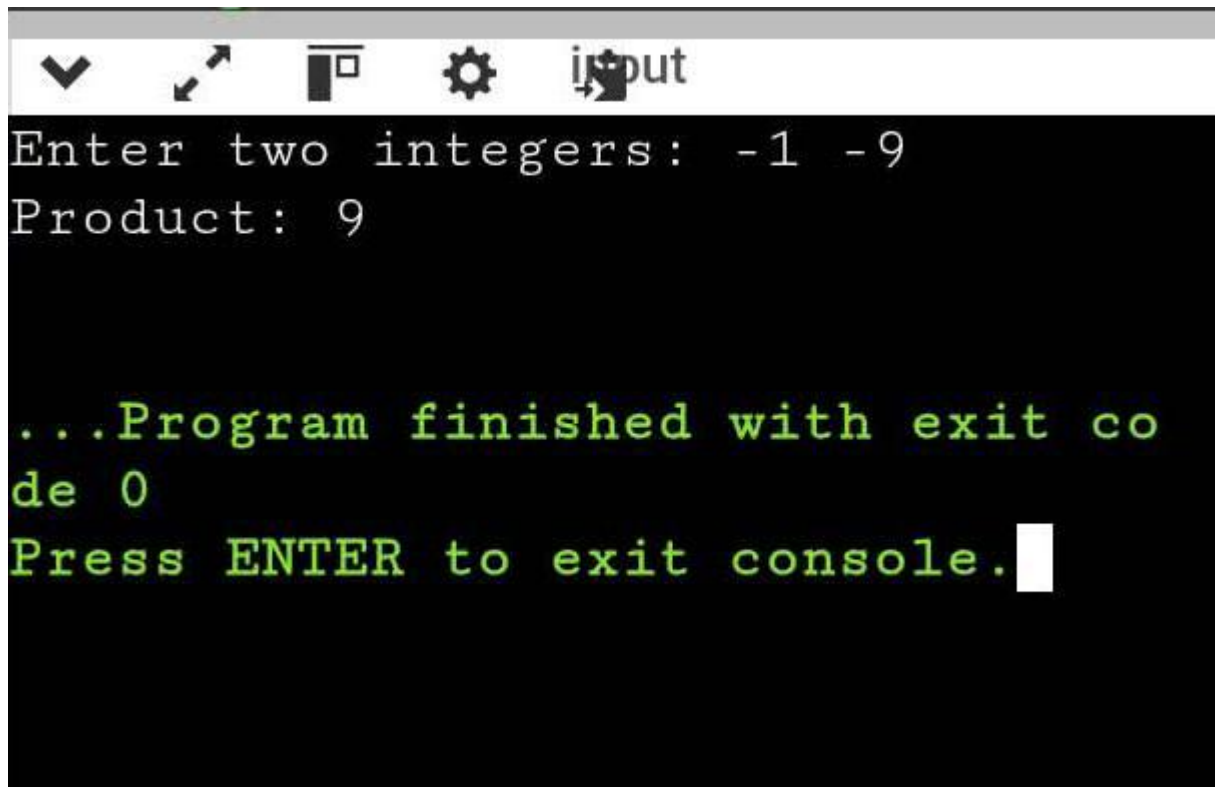
Smaller number is 1.

We add 9 only once.

$9 = 9$

Output: 9

Additions: 1

A screenshot of a console window with a dark background. The title bar at the top is light gray and contains several icons: a downward arrow, a square with an upward arrow, a square with a rightward arrow, a gear, and a cursor icon. The text in the console is white and green. The first two lines are white: "Enter two integers: -1 -9" and "Product: 9". The next two lines are green: "...Program finished with exit co" and "de 0". The final line is green: "Press ENTER to exit console." followed by a white cursor block.

```
Enter two integers: -1 -9
Product: 9

...Program finished with exit co
de 0
Press ENTER to exit console.
```

## Conclusion

The program gives the correct result for all the given test cases. It also handles positive numbers, negative numbers and zero correctly. The program does not use the `*` operator and uses the smaller number for repeated addition, so the number of additions is minimum.